VERSION CONTROLLING & WORKFLOWS WITH GIT - I

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
  - Describe the basic concepts of Version Controlling.

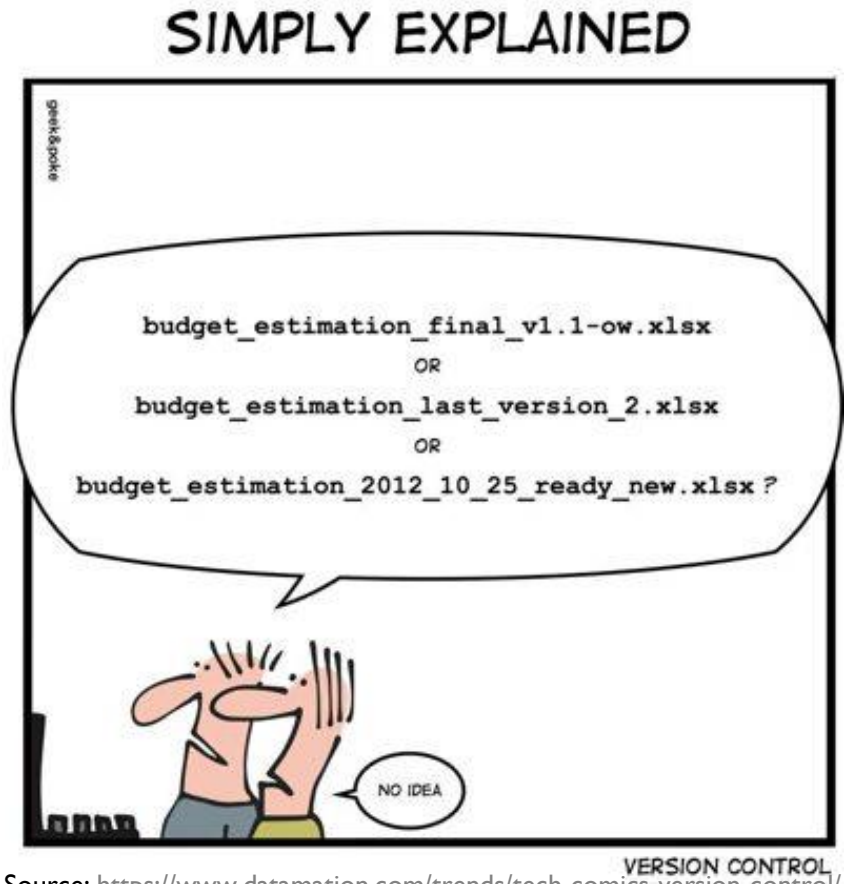
CONTENTS

- Version Controlling: What?
- What you know about Version Controlling
- Version Controlling: Scenario Discussion
- A bit of history on Version Controlling Systems (VCS)
- Git: the de-facto standard today in VCS
- Git Basic Concepts
- Conclusion

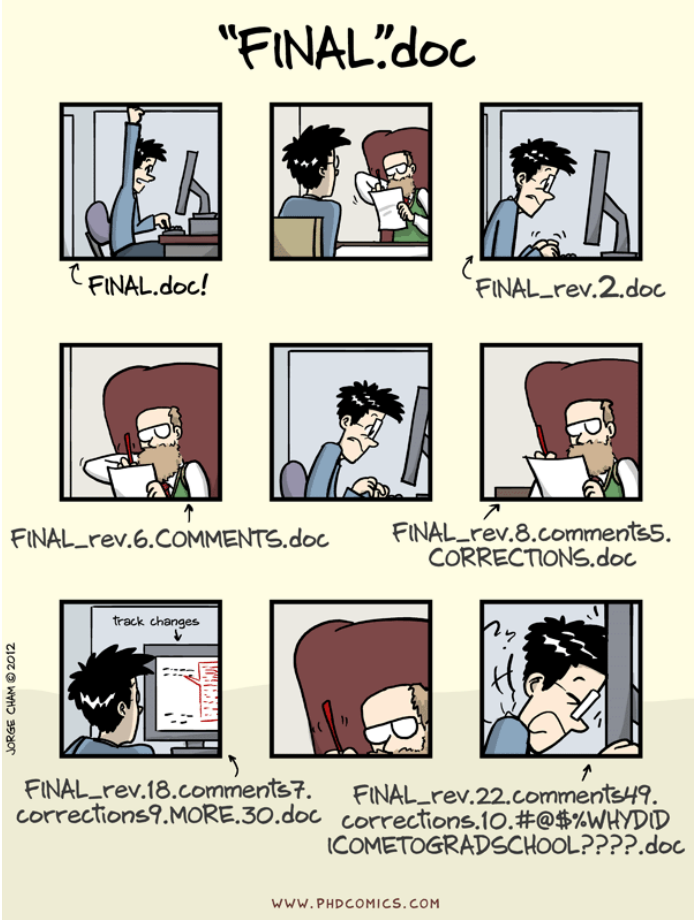
VERSION CONTROLLING: WHAT?

- “Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.” [1].

WHAT YOU KNOW ABOUT VERSION CONTROLLING



Source: https://www.datamation.com/trends/tech-comics-version-control/



Source: https://swcarpentry.github.io/hg-novice/01-basics/

WHAT YOU KNOW ABOUT VERSION CONTROLLING

- Q: What methods of Version Controlling are you familiar with?
  - Keeping multiple copies of files with different names
  - Google Drive/ OneDrive
  - Undo/ Redo Buffer

- Now, let's see how one student named Amith does version controlling.

VERSION CONTROLLING: SCENARIO DISCUSSION

- Suppose Amith is working on one file called numbers.py for an assignment.
- Amith has worked on the file for several days now, and has saved multiple copies of the file, i.e., numbers1.py, numbers2.py, numbers3.py and numbers.py.
- numbers.py is the latest version of the file. Whenever a new update is to be made to the file, first the existing numbers.py is renamed to numbersX.py (X = {1, 2, 3, 4…}) and then the numbers.py is saved with the most recent change.
- Amith needs to submit the assignment now, but at the last moment he discovers that the numbers.py (most recent version) throws a serious error.

VERSION CONTROLLING: SCENARIO DISCUSSION

- **Q1:** What can Amith do?
- **A:** Amith can submit the one before (e.g.: numbers10.py).

- **Q2:** What if the most recent version (numbers.py) has some good changes too, which Amith thinks needs to be in the submitted file?
- **A:**
  - Compare numbers10.py and numbers.py and Copy the good parts.
  - Paste the copied code as fit in the appropriate places in numbers10.py.
  - Replace the numbers.py with this modified file.

VERSION CONTROLLING: SCENARIO DISCUSSION

- The assignment was submitted. Now, Amith needs to keep this numbers.py and all its previous versions for future submissions too.
- However, Amith has a nagging feeling about what will happen if the HDD of his laptop fails.

- **Q3:** What can Amith do address this very valid concern?
- **A:** Copy all the versions of numbers.py to a Cloud storage location (OneDrive/ Google Drive).

VERSION CONTROLLING: SCENARIO DISCUSSION

- What are the pros of above approach?
  - All the versions are safely stored.
  - The files can be accessed from anywhere now.
- The cons?
  - Considering accessing from anywhere: If Amith develops two versions of numbers.py (both named the same), one at the computer lab in his university (then uploaded to the cloud) and another numbers.py separately on his laptop, Amith runs the risk of replacing the version on laptop with the other if he uploads the first file without the name being changed when his laptop syncs with the cloud storage.

VERSION CONTROLLING: SCENARIO DISCUSSION

- All in all, Amith now has a working Version Controlling System.
- His system can,
  - **Revert** to a past version.
  - **Compare** two file versions.
  - **Push** changes done from anywhere to a secure remote location on the cloud.
  - **Pull** changes from the cloud to anywhere.
  - **Merge** different versions of files.

VERSION CONTROLLING: SCENARIO DISCUSSION

- Now, Binesh joins Amith as the future assignments are to be done as pair work.
- It is more important than ever to follow a strict discipline when using Amith's system of version controlling as there are other people involved (than Amith).
- However, just focusing on naming the files correctly (to avoid overwrites) is not enough.
- Just seeing the files will not give each developer a high-level idea about what each other is doing.

VERSION CONTROLLING: SCENARIO DISCUSSION

- Solution?
  - A log file!
- How?
  - The log file should contain information on **who** did **what when** so everybody would be aware of what is happening.

Log - Notepad

File Edit Format View Help

```
| EntryNo | DateTime | Author | Change                                                                |
+---------+----------+--------+-----------------------------------------------------------------------+
| 01      | 08/02/23 | Amith  | Added numbers.py file and all previous versions to OneDrive.|
| 02      | 09/02/23 | Amith  | Added comments on Add() method on numbers.py.                         |
| 03      | 10/02/23 | Binesh | Added new file sequence.py to OneDrive folder.                        |
```

VERSION CONTROLLING: SCENARIO DISCUSSION

- Does Amith's method solve the problem of version controlling he had earlier?
  - Yes.
- Does that mean it is an easy system to follow?
  - Not really. There are several specific rules the users of the system must follow in order to make it work.
  - As the number of users of this system increase, so does the chance of someone making a mistake and ruining the system for everybody.
- Should he follow this approach then?
  - Yes, it's a good system. But the management of the system should not be given to the users. It should be automated.
- Are there such automated systems already?
  - Yes! That's what version controlling systems like GIT, SVN and Mercurial do.

A BIT OF HISTORY ON VERSION CONTROLLING SYSTEMS (VCS)

- The first proper Version Control System (VCS), named *Source Code Control System (SCCS)* was released in 1973 by Bell Labs.
- It was a first-generation VCS, which are now collectively known as **Local Version Control Systems**.
- Local VCS were intended to track changes for individual files and checked-out files could only be edited locally by one user at a time.
- Facilitating collaboration among large teams was problematic.

A BIT OF HISTORY ON VERSION CONTROLLING SYSTEMS (VCS)

- To address the problem of collaboration, a new type of VCS were developed.
- This second generation of VCS are collectively known as **Centralized Version Control Systems.**
- Centralized VCS allowed users to refer files stored on a centralized repository through a network and allowed multiple users to use the files concurrently.
- Follows a *Client-Server* approach.
- However, they would all have to commit their code to the centralized repository, which would require network access.
- A central authority is required to maintain the Centralized repository.

A BIT OF HISTORY ON VERSION CONTROLLING SYSTEMS (VCS)

- With the advent of open-source software came the third generation of VCS. They are collectively known as **Distributed Version Control Systems**.
- The earlier Centralized VCS required a central authority to maintain the centralized repository. No such authorities are usually found in open-source projects.
- Instead, there are just contributors who may develop features which may or may not be relevant to the main open-source project.
- Therefore, Distributed VCS have *peer-to-peer (P2P)* approach to version controlling.
- Distributed VCS are widely used today.
- De-facto standard in Distributed VCS now?
  - **Git!**

GIT: THE DE-FACTO STANDARD TODAY IN VCS

- Created by **Linus Torvalds** (creator of Linux) in 2005.
- Pronounced as *Ghi-T*. Not *Ji-T (*A "git" is a cranky old man. According to online sources, Linus meant himself).
- Originally designed to do version control on Linux kernel.
- Git supports non-linear development (thousands of parallel branches) and is fully distributed.

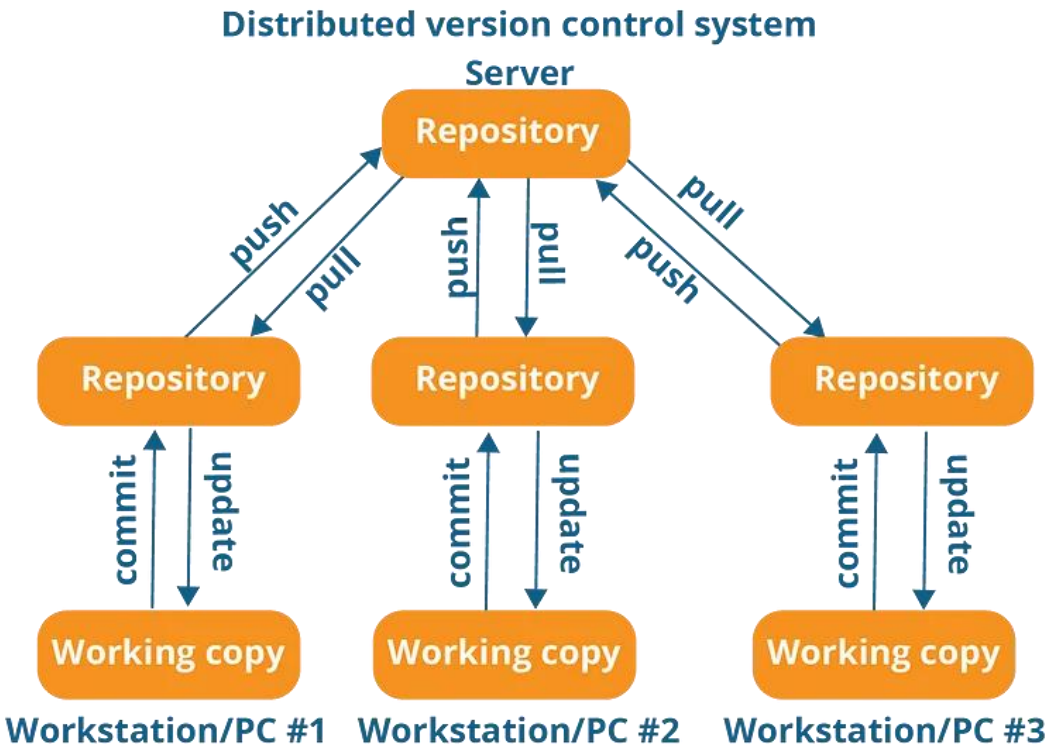
GIT BASIC CONCEPTS

- As mentioned, git does not necessarily need a Centralized Repository to work (It can work in somewhat centralized manner too, which we will see).
- Every user maintains a **local repository** of their own.
- When starting out they will obtain this local repository,
  - from another user or,
  - they may **clone** it from a central location or,
  - They may even initialize a brand-new repository from scratch.
- This repository will contain what others have done originally, what the user has done themselves and even work of others as selected by the user.
- They can **commit** and **update** their local repository without any interference from any others.

GIT CONCEPTS

- Git has two types of repositories.
  - Local repositories – which is owned by each user.
  - Remote repository – a centralized location accessible to all users. A single source of truth.
    - The users will **push** their new updates to this repository so those can be shared with others as well.
    - The users will **pull** new updates from others from this repository.
  - Why do we need Remote repositories?
    - Scenario Discussion: The problem with Distributed Version Controlling. Which project is the "right" project when multiple versions of the project exist?
    - Solution? Remote repositories as a single source of truth!
  - Git != GitHub (Git is not Github)!

GIT CONCEPTS



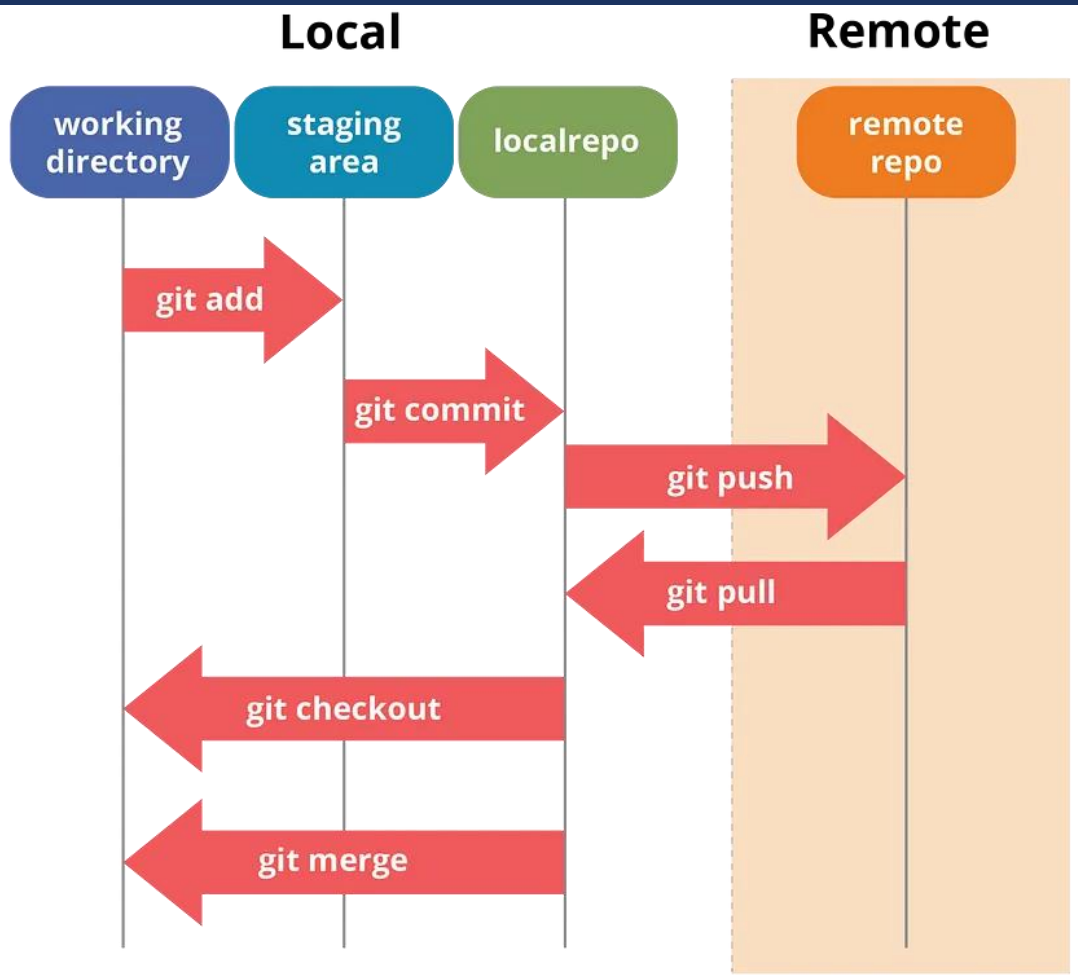
Source: https://medium.com/@sahoosunilkumar/how-does-git-works-5cc8444ea383

GIT CONCEPTS

- Git works on **Commits**.
- A Commit is very simply **a snapshot of all changes in the local repository** (more correctly, the working directory).
- Each Commit has a unique Commit ID.
- A Commit ID is a unique SHA-1 hash that is created whenever a new commit is recorded.
- The most recent commit is called the **Head** (more correctly, it refers to the currently checked-out branch's latest commit).
- For now, we will only deal with **main**/ master **Branch**.
- We will extensively deal with Commit IDs for many operations on git (more on this later).

GIT CONCEPTS

1. **Add** a new file to the local Repo.
2. Do the required changes to the file.
3. **Commit** the file to the local repo.
4. **Pull** the changes from the remote repo.
5. **Push** the local changes to the remote repo.



Source: https://medium.com/@sahoosunilkumar/how-does-git-works-5cc8444ea383

SCENARIO REVISITED

- Refer back to Amith's version controlling method in the beginning. It could,
  - **Revert** to a past version.
  - **Compare & Merge** two different versions of files.
  - **Pull** changes from a remote location to anywhere.
  - **Push** changes done from anywhere to a secure remote location.
  - Use the **Log file** to keep track of all the events.

- All these actions can be easily accomplished using Git.
  - Revert – git revert (this reverts a commit that was made to the repository).
  - Compare & Merge – Automatically handled by git. No user intervention required most of the time (unless a conflict occurs).
  - Push and Pull – Update the **Upstream** and get updates from upstream.
  - Git has many ways of automatically keeping track of all the events (e.g.: git status, log, blame etc.)

NEXT WEEK

- Branching
- Best Practices
- Pull Requests
- Git Workflows

REFERENCES

1. Pro Git - Scott Chacon and Ben Straub [Free eBook]
2. Dangit, Git!?! [Git quick reference]
3. Reading 5: Version Control [MIT Lecture]
4. Distributed Version Control [Wikipedia]
5. History of Version Control Systems

THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK